

Contents

01 - 入门	2
前提条件	3
构建步骤	3
项目初始化	3
第一个程序	3
注释的写法	4
02 - 变量与类型	4
变量声明	4
使用 @const 声明不可变变量	4
类型标注	4
基本类型	5
字符串操作	6
转义序列	6
重新赋值规则	7
元组解构	7
03 - 运算符	8
算術运算符	8
比較运算符	8
邏輯运算符	9
位元运算符	9
複合赋值运算符	9
遞增 / 遞減运算符	10
型別提升規則	10
歸屬运算符	11
控制流程	11
if / elif / else	11
while 迴圈	12
for 迴圈與 range	12
break 與 continue	13
巢狀範例	13
作用域規則	13
match	14
函式	15
基本函式定義	15
函式呼叫	15
遞迴函式	16
函式多載	16
省略回傳值型別 (Unit 型別)	16
結構體與列舉型別	16
使用 record 定義結構體	17
建構式的使用方式	17
欄位存取 (點記法)	17
欄位賦值	17
結構體作為函式參數	17
巢狀結構體	18
列舉型別 (enum)	18
集合	19

元组	19
列表	19
映射	22
集合	23
迭代器	23
高级特性	24
Lambda 函数	24
闭包	25
高阶函数	25
将函数作为值使用	25
UFCS (Uniform Function Call Syntax)	26
运算符重载	26
Option 类型	27
并发基础	27
网络 (TCP 套接字)	28
F-String (字符串插值)	29
类型转换 (as)	29
带关联数据的 enum (ADT)	29
泛型 enum	30
Result 类型	30
包	30
from/import 语法	31
子目录 (点分隔路径)	31
目录包	31
标准库 (std)	31
搜索路径优先级	32
PYTHONPATH 环境变量	32
限制	32
契约式设计	32
前置条件 (require)	32
后置条件 (ensure)	33
同时使用 require 和 ensure	33
结构体不变量 (invariant)	33
规则	34
测试	34
运行测试	34
编写测试	34
匹配器	35
输出格式	35
模拟 (Mock)	35
参数化测试	36
基于属性的测试	36
限制	36

[English](#) | [日本語](#) | [简体中文](#)

01 - 入门

下一篇教程 -> [02 - 变量与类型](#)

前提条件

要构建并运行 Ry，需要以下环境：

- LLVM 21
 - CMake 3.20 以上
 - 支持 C++17 的编译器（GCC 7+ / Clang 5+ 等）
-

构建步骤

在仓库根目录下执行以下命令：

```
cmake -B build -DLLVM_DIR=/usr/local/llvm/lib/cmake/llvm
cmake --build build
```

构建成功后，会生成 build/ry 可执行文件。

项目初始化

使用 ry new 命令创建新项目：

```
ry new my-project
cd my-project
```

这将生成以下文件和目录：

- package.toml – 项目配置文件
- src/main.ry – 入口点（附带示例代码）

若要将当前目录初始化为项目，请使用 ry init：

```
mkdir my-project
cd my-project
ry init
```

详情请参阅[项目管理](#)。

第一个程序

将以下内容保存为 hello.ry 文件：

```
print("Hello, World!")
```

使用以下命令运行：

```
./build/ry hello.ry
```

输出：

```
Hello, World!
```

也可以通过管道或 Here-document 从标准输入执行代码：

```
echo 'print("Hello, World!")' | ./build/ry
```

```
./build/ry <<'RY'
print("Hello, World!")
RY
```

注释的写法

从 # 到行尾的内容会被视为注释。

```
# 这是一段注释  
print("Hello") # 也可以在行尾添加注释
```

注释不会影响代码的执行。

下一篇教程 -> [02 - 变量与类型](#)

[English](#) | [日本語](#) | [简体中文](#)

02 - 变量与类型

<- [01 - 入门](#) / 下一篇-> [03 - 运算符](#)

变量声明

在 Ry 中，使用简单的赋值语法声明变量。默认情况下，变量是可变的。

```
x = 42          # 推断为 int  
y = 3.14        # 推断为 float  
flag = true     # 推断为 bool  
name = "Ry"     # 推断为 str
```

使用 @const 声明不可变变量

@const 指令将变量标记为不可变（常量）。声明后无法更改其值。

```
@const  
x = 42          # 推断为 int  
  
@const  
y = 3.14        # 推断为 float  
  
@const  
flag = true     # 推断为 bool  
  
@const  
name = "Ry"     # 推断为 str
```

类型标注

可以显式指定变量的类型。

```
x: int = 42  
  
rate: float = 0.5
```

```
ok: bool = false
```

```
msg: str = "hello"
```

当类型标注与实际值的类型不一致时，会产生编译错误。

基本类型

类型	说明	字面值示例
int	64 位整数	0, 42, -10
u8	无符号 8 位整数 (0-255)	b: u8 = 42
float	64 位浮点数	0.0, 3.14, -1.5
bool	布尔值	true, false
str	字符串	"hello", ""

底层数值类型

Ry 还提供底层数值类型，用于精确控制内存布局。这些类型没有隐式转换——必须使用 `as` 进行显式转换。

类型	说明	示例
i8	8 位有符号整数	x: i8 = 42
i16	16 位有符号整数	x: i16 = 100
i32	32 位有符号整数	x: i32 = 42
i64	64 位有符号整数	x: i64 = 100
u8	8 位无符号整数	x: u8 = 200
u16	16 位无符号整数	x: u16 = 60000
u32	32 位无符号整数	x: u32 = 3000000000
u64	64 位无符号整数	x: u64 = 100
f32	32 位浮点数	x: f32 = 3.14

```
a: i32 = 10
b: i32 = 20
c = a + b           # OK: i32 + i32 → i32

d = 42
# e = a + d         # Error: cannot mix i32 and int

y = a as int        # 显式转换为 int
z = d as i32        # 显式转换为 i32

# 无符号类型使用无符号运算
x: u32 = 3000000000
y: u32 = 7
q = x / y           # 无符号除法 (UDiv)
```

注意: 底层整数的 `/` 执行整数除法 (类似 Rust)，而非浮点除法。有符号类型使用 `SDiv`，无符号类型使用 `UDiv`。

注意: 底层整数的算术运算在溢出时会回绕。有符号类型使用二进制补码，无符号类型使用模运算。如果担心溢出，请使用高级 `int` 类型 (64 位)。

固定长度数组

对于底层类型，Ry 提供固定长度连续数组 [T; N]。这些数组在栈上分配，大小在编译时确定。

```
buf: [i32; 4] = [1, 2, 3, 4]
print(buf[0])      # 1
buf[2] = 99
print(buf[2])      # 99
print(length(buf)) # 4

pixels: [u8; 3] = [255, 128, 0]
```

字符串操作

字符串支持多种操作。

```
a = "Hello"
b = "World"

# 拼接
c = a + ", " + b    # "Hello, World"

# 比较 (字典序)
print(a == b)      # false
print(a != b)      # true
print(a < b)       # true ("H" < "W")

# 长度
print(length(a))   # 5

# 子字符串检查
s = "Hello, World!"
print(contains(s, "World"))    # true
print(starts_with(s, "Hello")) # true
print(ends_with(s, "!"))      # true
```

转义序列

字符串中可使用以下转义序列。

序列	含义
\n	换行
\r	回车
\t	Tab
\\	反斜杠
\"	双引号
\0	空字符

```
print("Hello\nWorld") # 分成两行输出
print("A\tB")         # 以 Tab 分隔
print("say \"hi\"")    # 包含双引号的字符串
```

重新赋值规则

未使用 `@const` 声明的变量可以重新赋值，但有以下限制：

```
x = 10
x = 20      # OK: 重新赋予相同类型的值
# x = "text" # 错误：禁止更改类型的重新赋值
```

`@const` 变量无法重新赋值。

```
@const
N = 5
# N = 6 # 错误：禁止对 @const 变量重新赋值
```

也无法重新声明同名的变量。

```
x = 1
# 同一作用域内禁止重新声明同名变量
```

元组解构

可以在单次声明中将元组拆解为多个变量。

```
@const
a, b = (10, 20)
print(a)  # 10
print(b)  # 20
```

通配符

使用 `_` 忽略特定位置的值。

```
@const
x, _ = (1, 2)  # 只绑定 x; 2 被丢弃
print(x)      # 1
```

可变变量解构

省略 `@const` 即可声明可变变量。

```
a, b = (10, 20)
a = 99
print(a)  # 99
```

规则

- 左侧的变量数量必须与元组的元素数量相符。
 - 每个变量遵循与普通声明相同的 `@const`/可变规则。
 - 不支持嵌套元组解构。
-

[<- 01 - 入门](#) / [下一篇-> 03 - 运算符](#)

[English](#) | [日本語](#) | [繁體中文](#)

03 - 運算子

← [02 - 變數與型別](#) / 下一個 → [04 - 控制流程](#)

算術運算子

運算子	說明	範例	結果
+	加法	3 + 2	5
-	減法	3 - 2	1
*	乘法 / 字串重複	3 * 2	6
/	除法 (始終為 float)	7 / 2	3.5
//	整數除法 (始終為 int)	7 // 2	3
%	取餘	7 % 3	1
**	次方 (始終為 float)	2 ** 10	1024.0

```
a = 10
b = 3

print(a + b)    # 13
print(a - b)    # 7
print(a * b)    # 30
print(a / b)    # 3.3333... (float)
print(a // b)   # 3 (int)
print(a % b)    # 1
print(2 ** 8)   # 256.0 (float)
```

比較運算子

比較運算子皆回傳 bool 值。

運算子	說明	範例
==	等於	a == b
!=	不等於	a != b
<	小於	a < b
<=	小於等於	a <= b
>	大於	a > b
>=	大於等於	a >= b

```
x = 5
y = 10

print(x == y)   # false
print(x != y)   # true
print(x < y)    # true
print(x <= y)   # true
print(x > y)    # false
print(x >= y)   # false
```

字串也可使用比較運算子（字典序比較）。


```
print("abc" == "abc")    # true
print("abc" < "abd")     # true
print("b" > "a")         # true
```

邏輯運算子

運算子	說明	範例
and	邏輯 AND	a and b
or	邏輯 OR	a or b
not	邏輯 NOT	not a

```
t = true
f = false
```

```
print(t and f)    # false
print(t or f)     # true
print(not t)      # false
print(not f)      # true
```

位元運算子

位元運算子僅適用於 `int` 型別。

運算子	說明	範例
&	位元 AND	5 & 3 → 1
	位元 OR	5 3 → 7
^	位元 XOR	5 ^ 3 → 6
~	位元 NOT (一元)	~5 → -6
<<	左移	1 << 3 → 8
>>	算術右移	8 >> 2 → 2
>>>	邏輯右移	-1 >>> 1 → 9223372036854775807

```
a = 0b1010    # 10
b = 0b1100    # 12
```

```
print(a & b)    # 8 (0b1000)
print(a | b)    # 14 (0b1110)
print(a ^ b)    # 6 (0b0110)
print(~a)       # -11
print(1 << 4)   # 16
print(32 >> 2)  # 8
```

複合賦值運算子

更新變數值時可使用的簡寫語法。

運算子	說明	等同的表達式
+=	加法賦值	x = x + n
-=	減法賦值	x = x - n
*=	乘法賦值	x = x * n
/=	除法賦值	x = x / n
%=	取餘賦值	x = x % n

```
x = 10
x += 5    # x == 15
x -= 3    # x == 12
x *= 2    # x == 24
x /= 4    # x == 6.0 (會變為 float)
```

遞增／遞減運算子

將變數增減 1 的簡寫語法。

運算子	說明	等同的表達式
x++	加 1	x = x + 1
x--	減 1	x = x - 1

```
count = 0
count++    # count == 1
count++    # count == 2
count--    # count == 1
```

注意：僅可作為陳述式使用，不能在運算式中使用。

型別提升規則

以下說明運算中 int 與 float 混合時的行為。

```
# + - * 當其中一方為 float 時結果為 float
print(1 + 2)      # 3 (int)
print(1 + 2.0)    # 3.0 (float)
print(1.0 + 2)    # 3.0 (float)

# / 始終為 float
print(4 / 2)      # 2.0 (float)

# // 始終為 int
print(7 // 2)     # 3 (int)
print(7.0 // 2)   # 3 (int)

# ** 始終為 float
print(2 ** 3)     # 8.0 (float)

# % 當兩邊皆為 int 時為 int，其中一方為 float 時為 float
print(7 % 3)      # 1 (int)
print(7.5 % 2)    # 1.5 (float)
```

```
# + 當兩邊皆為 str 時為字串串接
print("foo" + "bar")    # "foobar"

# * 當一方為 str、另一方為 int 時為字串重複
print("ab" * 3)         # "ababab"
print(3 * "ab")         # "ababab"
```

歸屬運算子

運算子	說明	範例
in	歸屬檢查	2 in {1, 2, 3} → true
not in	否定歸屬檢查	4 not in {1, 2, 3} → true

```
s = {1, 2, 3}
print(2 in s)      # true
print(4 not in s)  # true
```

[← 02 - 變數與型別](#) / [下一個 → 04 - 控制流程](#)

[English](#) | [日本語](#) | [繁體中文](#)

控制流程

[← 前一篇：運算子](#) | [下一篇：函式](#) →

if / elif / else

使用 if 根據條件進行分支處理。

```
x = 10
```

```
if x > 0:
    print(x)
elif x == 0:
    print(0)
else:
    print(-1)
```

- elif 和 else 可以省略。
- 條件式不限於 bool，也可以指定其他型別。對於 int，0 視為假，非 0 視為真。
- if 可以巢狀使用。

```
a = 5
b = 3
```

```
if a > 0:
    if b > 0:
        print(a + b)    # 8
```

while 迴圈

當條件為真時，重複執行區塊。

```
i = 3
while i > 0:
    print(i)
    i = i - 1
# 3
# 2
# 1
```

for 迴圈與 range

可使用列表或 range 進行迭代。

```
for x in [1, 2, 3]:
    print(x)
# 1
# 2
# 3
```

range(n) 產生從 0 到 n - 1 的整數。

```
for i in range(5):
    print(i)
# 0
# 1
# 2
# 3
# 4
```

range(start, end) 產生從 start 到 end - 1 的整數。

```
for i in range(2, 5):
    print(i)
# 2
# 3
# 4
```

.. 範圍運算子建立包含兩端的範圍。1 .. 3 產生 [1, 2, 3]。

```
for i in 1 .. 3:
    print(i)
# 1
# 2
# 3
```

使用 for k, v in map 可以走訪映射的鍵值對。

```
m = {"x": 10, "y": 20}
for k, v in m:
    print(k)
    print(v)
```

break 與 continue

break 立即跳出迴圈。continue 跳過目前的迭代，進入下一次迭代。

```
for i in range(10):
    if i == 5:
        break
    if i % 2 == 0:
        continue
    print(i)
# 1
# 3
```

在 while 中也可同樣使用。

```
n = 0
while true:
    n = n + 1
    if n % 2 == 0:
        continue
    if n > 7:
        break
    print(n)
# 1
# 3
# 5
# 7
```

注意：在巢狀迴圈中，break / continue 僅作用於最內層的迴圈。在迴圈外使用會產生編譯錯誤。

巢狀範例

for 和 while 可以巢狀使用。

```
for i in range(1, 4):
    for j in range(1, 4):
        if j == 2:
            continue
        print(i * 10 + j)
# 11
# 13
# 21
# 23
# 31
# 33
```

作用域規則

控制流程的區塊具有作用域。

區塊作用域

在區塊內宣告的變數無法從區塊外部參照。

```

if true:
    inner = 42
# 在此處參照 inner 會產生編譯錯誤

```

參照與重新賦值外部變數

可以從區塊內參照和重新賦值外部的變數。

```

count = 0
for i in range(5):
    count = count + i
print(count)    # 10

```

內層作用域的重新賦值

在區塊內對變數賦值會修改外層的變數（Python 風格的作用域）。不會產生遮蔽——內層的賦值會修改同一個變數。

```

x = 1
if true:
    x = 99
    print(x)    # 99
print(x)        # 99

```

match

match 是根據值進行分支的語法，可安全地處理 enum 和 Option。

```

enum Color:
    Red
    Green
    Blue

c = Color::Green
match c:
    case Color::Red:
        print("red")
    case Color::Green:
        print("green")
    case Color::Blue:
        print("blue")
# green

```

Option 的匹配

使用 match 可以安全地處理 None 的情況。

```

x: Option<int> = Some(42)
match x:
    case Some(v):
        print(v)
    case None:
        print("nothing")
# 42

```

萬用字元與字面值

`_` 是可匹配任何值的萬用字元模式。也可以使用字面值（數值、字串、布林值）進行匹配。

```
n = 5
match n:
    case 0:
        print("zero")
    case 1:
        print("one")
    case _:
        print("other")
# other
```

guard 子句

可以使用 `if` 新增守衛條件。

```
match n:
    case x if x > 0:
        print("positive")
    case x if x < 0:
        print("negative")
    case _:
        print("zero")
```

注意：match 必須涵蓋所有模式。enum 必須包含所有變體，Option 必須包含 Some 和 None，字面值則需要 `_`。

[← 前一篇：運算子](#) | [下一篇：函式](#) →

[English](#) | [日本語](#) | [繁體中文](#)

函式

[← 前一篇：控制流程](#) | [下一篇：結構體](#) →

基本函式定義

函式使用 `fn` 關鍵字定義。參數的型別宣告為必要項，以 `name: type` 的格式指定。回傳值型別在 `->` 之後指定。

```
fn add(a: int, b: int) -> int:
    return a + b
```

- 參數的型別宣告是必要的。
 - 回傳值型別在 `->` 之後指定。
 - 使用 `return` 陳述式回傳值。
-

函式呼叫

透過名稱和參數來呼叫已定義的函式。

```
fn multiply(x: int, y: int) -> int:
    return x * y
```

```
result = multiply(3, 4)
print(result)    # 12
```

遞迴函式

函式可以呼叫自身（遞迴）。

```
fn factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))    # 120
print(factorial(0))    # 1
```

函式多載

可以定義多個同名但參數數量或型別不同的函式。

```
fn add(a: int, b: int) -> int:
    return a + b

fn add(a: float, b: float) -> float:
    return a + b

print(add(1, 2))        # 3
print(add(1.5, 2.5))    # 4
```

呼叫時會根據參數的型別自動選擇適當的函式。

注意：若定義參數型別相同但僅回傳值型別不同的函式，會產生編譯錯誤。

省略回傳值型別（Unit 型別）

不需要回傳值的函式可以省略 `->`。此時函式回傳 Unit 型別。

```
fn greet():
    print(42)

greet()    # 42
```

這是最簡單的無參數、無回傳值函式形式。

[← 前一篇：控制流程](#) | [下一篇：結構體](#) →

[English](#) | [日本語](#) | [繁體中文](#)

結構體與列舉型別

[← 前一篇：函式](#) | [下一篇：集合](#) →

使用 record 定義結構體

使用 record 關鍵字定義結構體。各欄位以 name: type 的格式描述。

```
record Point:
  x: int
  y: int
```

結構體是堆疊上的值型別。

建構式的使用方式

像呼叫函式一樣使用結構體名稱來產生實例。參數按照欄位的定義順序指定。

```
p = Point(10, 20)
```

欄位存取（點記法）

使用點記法存取欄位。

```
p = Point(10, 20)
print(p.x)    # 10
print(p.y)    # 20
```

注意：將結構體直接傳給 print 會產生錯誤。請個別傳遞欄位。

欄位賦值

未使用 @const 宣告的可變變數的欄位可以重新賦值。

```
p = Point(10, 20)
p.x = 100
print(p.x)    # 100
```

注意：對 @const 宣告的變數的欄位賦值會產生編譯錯誤。

結構體作為函式參數

可以將結構體作為函式的參數傳遞。

```
record Point:
  x: int
  y: int

fn distance_x(a: Point, b: Point) -> int:
  return a.x - b.x

p1 = Point(10, 3)
p2 = Point(4, 7)
print(distance_x(p1, p2))    # 6
```

巢狀結構體

結構體的欄位可以使用其他結構體。

```
record Point:
    x: int
    y: int

record Line:
    start: Point
    end: Point

line = Line(Point(0, 0), Point(10, 5))
print(line.start.x)    # 0
print(line.end.x)      # 10
```

透過鏈結點記法可存取巢狀欄位。

列舉型別 (enum)

使用 `enum` 關鍵字定義列舉型別。每個變體作為具名常數處理。

定義

```
enum Color:
    Red
    Green
    Blue
```

使用方式

使用 `::` 存取變體。

```
c = Color::Red
print(c)    # Red
```

比較

可以使用 `==` 和 `!=` 比較變體。

```
if c == Color::Red:
    print("red!")
elif c == Color::Green:
    print("green!")
else:
    print("blue!")
```

函式參數

可以使用 `enum` 名稱作為函式的參數型別。

```
fn describe(c: Color) -> str:
    if c == Color::Red:
        return "warm"
    return "cool"
```

[← 上一篇：函式](#) | [下一篇：集合](#) →

[English](#) | [日本語](#) | [简体中文](#)

集合

[<- 上一篇：结构体](#) | [下一篇：高级特性 ->](#)

Ry 有四种集合类型：元组、列表、映射、集合。

元组

元组是将多个值组合在一起的不可变数据结构，可以持有不同类型的元素。

创建

```
t = (1, 3.14)
```

类型标注

```
t: (int, float) = (1, 3.14)
```

元素访问

使用 `.0`、`.1` 等索引来访问元素。

```
t = (1, 3.14)
print(t.0)    # 1
print(t.1)    # 3.14
```

作为函数返回值

想要返回多个值时，元组很方便。

```
fn swap(a: int, b: int) -> (int, int):
    return (b, a)
```

```
result = swap(1, 2)
print(result.0) # 2
print(result.1) # 1
```

限制

- 超出范围的索引（例如：对只有 2 个元素的元组使用 `.2` 访问）会产生编译错误。
 - 将元组直接传给 `print` 会产生错误。请逐个传递各元素。
-

列表

列表是由相同类型的元素组成的可变长度数据结构。

创建

```
xs = [1, 2, 3]
```

类型标注

```
xs: List<int> = [1, 2, 3]
```

索引访问

```
print(xs[0])    # 1
```

```
i = 1
print(xs[i])    # 2
```

索引赋值

```
xs[0] = 99
```

length

```
print(length(xs))    # 3
```

print

```
print(xs)    # [1, 2, 3]
```

for 遍历

```
for x in xs:
    print(x)
```

函数参数

```
fn first(xs: List<int>) -> int:
    return xs[0]
```

filter、map、sort

列表支持 filter、map、sort 操作。这些操作返回新列表，不会修改原始列表。

```
xs = [1, 2, 3, 4, 5]
```

```
# filter: 保留符合条件的元素
```

```
evens = xs.filter(fn(x: int) => x > 3)
print(evens)    # [4, 5]
```

```
# map: 转换每个元素
```

```
doubled = xs.map(fn(x: int) => x * 2)
print(doubled)    # [2, 4, 6, 8, 10]
```

```
# sort: 升序排序（默认）
```

```
sorted = [3, 1, 2].sort()
print(sorted)    # [1, 2, 3]
```

```
# 链式调用
```

```
result = xs.filter(fn(x: int) => x > 1).map(fn(x: int) => x * 10).sort()
print(result)    # [20, 30, 40, 50]
```

reduce、fold

`reduce` 从第一个元素开始将列表归约为单一值。`fold` 则可提供明确的初始值。

```
xs = [1, 2, 3, 4, 5]
```

```
# reduce: 从第一个元素开始
```

```
total = reduce(xs, fn(a: int, b: int) => a + b)
print(total)    # 15
```

```
# fold: 提供明确的初始值
```

```
total2 = fold(xs, 0, fn(a: int, b: int) => a + b)
print(total2)   # 15
```

any、all

`any` 如果至少有一个元素满足谓词则返回 `true`。`all` 如果所有元素都满足则返回 `true`。

```
xs = [1, 2, 3, 4, 5]
```

```
print(any(xs, fn(x: int) => x > 4))    # true
print(any(xs, fn(x: int) => x > 9))    # false
```

```
print(all(xs, fn(x: int) => x > 0))    # true
print(all(xs, fn(x: int) => x > 3))    # false
```

sum、min、max

```
xs = [3, 1, 4, 1, 5]
print(sum(xs))    # 14
print(min(xs))    # 1
print(max(xs))    # 5
```

first、last、is_empty

```
xs = [10, 20, 30]
print(first(xs))    # 10
print(last(xs))     # 30
print(is_empty(xs)) # false
```

enumerate、zip

`enumerate` 为每个元素附上索引。`zip` 将两个列表逐元素合并。

```
xs = [10, 20, 30]
indexed = enumerate(xs)
# [(0, 10), (1, 20), (2, 30)]
for p in indexed:
    print(p.0)
    print(p.1)

ys = ["a", "b", "c"]
zipped = zip(xs, ys)
# [(10, "a"), (20, "b"), (30, "c")]
```

限制

- 所有元素必须是相同类型，不能混合不同类型。

- 空列表 `[]` 会产生错误。
 - 超出范围的访问会产生运行时错误 (`exit(1)`)。
 - 元素类型支持 `int`、`float`、`bool`、`str`。
-

映射

映射是管理键值对的关联数组。

创建

```
m = {"a": 1, "b": 2}
```

类型标注

```
m: Map<str, int> = {"a": 1, "b": 2}
```

键访问

```
print(m["a"])    # 1
```

插入 / 更新

对新的键赋值即为插入，对已有的键赋值即为更新。

```
m["c"] = 3      # 插入新条目
m["a"] = 99     # 更新已有条目
```

length

```
print(length(m))    # 3
```

print

```
print(m)    # {a: 99, b: 2, c: 3}
```

has_key

检查键是否存在。

```
print(m.has_key("a"))    # true
```

keys、values

`keys` 返回所有键的列表。`values` 返回所有值的列表。

```
m = {"a": 1, "b": 2, "c": 3}
print(keys(m))    # ["a", "b", "c"]
print(values(m))  # [1, 2, 3]
```

函数参数

```
fn get_val(m: Map<str, int>, k: str) -> int:
    return m[k]
```

限制

- 所有键必须是相同类型，所有值也必须是相同类型。
 - 空映射需要类型标注。
 - 访问不存在的键会产生运行时错误 (`exit(1)`)。
-

集合

集合是持有相同类型元素且不重复的集合类型。

创建

```
s = {1, 2, 3}
```

类型标注

```
s: Set<int> = {1, 2, 3}
```

in 运算符

使用 `in` 运算符检查元素是否包含在集合中。

```
print(2 in s)    # true
print(5 in s)    # false
```

add / remove

```
s.add(4)         # 添加元素
s.remove(1)      # 移除元素
s.add(2)         # 已存在，因此忽略
```

length / print

```
print(length(s)) # 3
print(s)         # {2, 3, 4}
```

for 遍历

```
for x in s:
    print(x)
```

空集合

空集合需要类型标注。

```
empty: Set<int> = {}
```

限制

- 所有元素必须是相同类型。
 - 元素类型支持 `int`、`float`、`bool`、`str`。
-

迭代器

迭代器提供一种惰性的方式来处理集合。迭代器不会在每个步骤中创建中间列表，而是通过管道逐个处理元素。

创建和消费

```
xs = [1, 2, 3]
ys = xs.iter().to_list()    # [1, 2, 3]
```

链式操作

可以链式调用 `filter`、`map` 和 `take` 来构建管道：

```
result = [1, 2, 3, 4, 5]
    .iter()
    .filter(fn(x: int) => x > 2)
    .map(fn(x: int) => x * 2)
    .take(2)
    .to_list()
print(result)    # [6, 8]
```

使用 `next()` 手动迭代

```
it = [10, 20].iter()
print(it.next())    # Some(10)
print(it.next())    # Some(20)
print(it.next())    # None
```

For 循环

迭代器可以直接在 `for` 循环中使用：

```
for x in [1, 2, 3].iter().filter(fn(x: int) => x > 1):
    print(x)    # 2, 3
```

映射产生元组元素：

```
for k, v in {"a": 1, "b": 2}.iter():
    print(k)
```

[<- 上一篇：结构体](#) | [下一篇：高级特性](#) ->

[English](#) | [日本語](#) | [简体中文](#)

高级特性

[<- 上一篇：集合](#) | [下一篇：包](#) ->

Lambda 函数

Lambda 函数是将函数以表达式形式编写的语法，以 `fn(参数) => 表达式` 的形式书写。返回值类型会自动推断。

单一表达式 Lambda

```
double = fn(x: int) => x * 2
print(double(5))    # 10
```

```
add = fn(a: int, b: int) => a + b
print(add(3, 4))    # 7
```


无参数 Lambda

```
answer = fn() => 42
print(answer()) # 42
```

多行 Lambda

在 `:` 后换行并缩进，即可编写多条语句。

```
abs = fn(x: int):
    if x < 0:
        return -x
    return x

print(abs(-5)) # 5
print(abs(3)) # 3
```

闭包

Lambda 函数可以捕获定义时作用域中的变量。

```
offset = 10
add_offset = fn(x: int) => x + offset
print(add_offset(5)) # 15
```

高阶函数

可以定义接受函数作为参数的函数。函数类型以 `fn(参数类型) -> 返回值类型` 的形式书写。

```
fn apply(f: fn(int) -> int, x: int) -> int:
    return f(x)

double = fn(x: int) => x * 2
print(apply(double, 3)) # 6
print(apply(fn(n: int) => n + 1, 10)) # 11
```

将函数作为值使用

具名函数也可以绑定到变量或作为参数传递。

```
fn square(x: int) -> int:
    return x * x

fn apply(f: fn(int) -> int, x: int) -> int:
    return f(x)

# 将具名函数作为参数传递
print(apply(square, 4)) # 16

# 绑定到变量
sq = square
print(sq(5)) # 25
```

UFCS (Uniform Function Call Syntax)

使用 UFCS 可以将 `f(a, b)` 的调用写成 `a.f(b)`，实现类似方法链的写法。

```
fn add(a: int, b: int) -> int:
    return a + b
```

```
x = 1
print(x.add(2))    # add(x, 2) -> 3
```

链式调用

```
fn double(n: int) -> int:
    return n * 2

print(x.add(2).double())    # double(add(x, 2)) -> 6
```

运算符重载

使用 `fn operator` 语法可为自定义类型定义运算符。

二元运算符

接受 2 个参数。

```
record Vec2:
    x: int
    y: int

fn operator+(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)

fn operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y
```

```
v1 = Vec2(1, 2)
v2 = Vec2(3, 4)
v3 = v1 + v2
print(v3.x)    # 4
print(v1 == v2)    # false
```

一元运算符

接受 1 个参数。

```
fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

支持的运算符

类别	运算符
算术	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> , <code>**</code> , <code>//</code>

类别	运算符
比较	==, !=, <, <=, >, >=
位运算	&, \ , ^, ~, <<, >>
逻辑	and, or, not

Option 类型

表示值是否存在的类型，可以是 Some(值) 或 None。

```
x: Option<int> = Some(42)
print(x)      # Some(42)
```

```
y: Option<int> = None
print(y)      # None
```

取出值

使用 match 安全地取出内部的值，并处理 None 的情况。

```
match x:
    case Some(v):
        print(v)      # 42
    case None:
        print("nothing")
```

并发基础

Task<T> 是并发工作的运行时的句柄。使用 async fn 定义返回任务的函数，在另一个 async fn 内部使用 await，从同步上下文使用 block_on(task) 等待完成。

```
async fn add(a: int, b: int) -> int:
    return a + b
```

```
# 从同步上下文使用 block_on()
t: Task<int> = add(20, 22)
print(block_on(t))      # 42
print(block_on(add(1, 2)))  # 3
```

```
# 在 async fn 内部使用 await
async fn double_add(a: int, b: int) -> int:
    return (await add(a, b)) * 2
```

@parallel 可以应用于使用 range(...) 或整数 .. 范围的计数 for 循环：

```
@parallel
for i in range(8):
    print(i)
```

@parallel for 禁止使用 break、continue 以及对外部可变变量的写入。

网络 (TCP 套接字)

Ry 通过 `net` 模块提供 TCP 套接字支持。`send`、`recv` 和 `close` 函数用于 TCP 套接字操作。所有网络操作返回 `Result` 类型。

```
from net import bind, listen, accept, connect, listener_port
from io import str_to_bytes, bytes_to_str

async fn echo_server(server: TcpListener) -> str:
    match accept(server):
        case Ok(conn):
            match recv(conn, 4096):
                case Ok(data):
                    match send(conn, data):
                        case Ok(_):
                            ...
                        case Err(e):
                            ...
                    case Err(e):
                        ...
            close(conn)
        case Err(e):
            ...
    close(server)
    return "done"

match bind("127.0.0.1", 0):
    case Ok(server):
        match listen(server, 1):
            case Ok(_):
                port = listener_port(server)
                t = echo_server(server)
                match connect("127.0.0.1", port):
                    case Ok(conn):
                        match send(conn, str_to_bytes("hello")):
                            case Ok(_):
                                ...
                            case Err(e):
                                ...
                        match recv(conn, 4096):
                            case Ok(resp):
                                match bytes_to_str(resp):
                                    case Ok(s):
                                        print(s)    # hello
                                    case Err(e):
                                        ...
                                case Err(e):
                                    ...
                            case Err(e):
                                ...
                        close(conn)
                    case Err(e):
                        print("connect failed")
                block_on(t)
            case Err(e):
                ...
    case Err(e):
```

```
print("bind failed")
```

完整 API 请参阅[网络参考手册](#)。

F-String (字符串插值)

使用 `f"..."` 可以在字符串中直接嵌入表达式。表达式放在 `{}` 内。

```
name = "Alice"
print(f"Hello {name}")    # Hello Alice

x = 3
y = 4
print(f"{x} + {y} = {x + y}")    # 3 + 4 = 7
```

使用 `{{` 和 `}}` 来包含字面大括号。

```
print(f"{{escaped}}")    # {escaped}
```

类型转换 (as)

使用 `as` 在类型之间进行显式转换。

```
x = 42 as float    # 42.0
y = 3.14 as int    # 3 (截断)
s = 42 as str      # "42"
b = true as int    # 1
```

带关联数据的 enum (ADT)

`enum` 变体可以携带关联值。这使得单一 `enum` 可以表示一系列不同形状的数据。可以选择性地为字段命名以提高文档清晰度。

```
enum Shape:
    Circle(radius: float)
    Rectangle(width: float, height: float)
    Point
```

命名字段仅用于文档说明——使定义具有自描述性。无名语法 (`Circle(float)`) 同样有效。

构造 ADT 变体

构造始终按位置进行，无论字段是否命名。

```
c = Shape::Circle(3.14)
r = Shape::Rectangle(4.0, 5.0)
p = Shape::Point
```

匹配 ADT 变体

在 `case` 中使用绑定模式来提取关联数据。绑定使用你选择的变量名，而非字段名。

```
fn describe(s: Shape) -> str:
    match s:
        case Shape::Circle(r):
```

```

        return f"circle with radius {r}"
    case Shape::Rectangle(w, h):
        return f"rectangle {w}x{h}"
    case Shape::Point:
        return "point"

print(describe(Shape::Circle(3.14)))      # circle with radius 3.14
print(describe(Shape::Rectangle(4.0, 5.0))) # rectangle 4.0x5.0

```

泛型 enum

enum 可以带有类型参数，使其可在不同载荷类型间重复使用。

```

enum MyOption<T>:
    MySome(T)
    MyNone

```

用法

```

a = MyOption<int>::MySome(42)
b: MyOption<int> = MyOption<int>::MyNone

```

```

match a:
    case MyOption::MySome(v):
        print(v)      # 42
    case MyOption::MyNone:
        print("none")

```

Result 类型

Result<T, E> 用于可能失败的函数。成功时返回 Ok(value)，失败时返回 Err(error)。

```

fn divide(a: int, b: int) -> Result<int, str>:
    if b == 0:
        return Err("division by zero")
    return Ok(a // b)

```

使用 match 来处理结果。

```

r = divide(10, 0)
match r:
    case Ok(v):
        print(v)
    case Err(e):
        print(e)      # division by zero

```

[<- 上一篇：集合](#) | [下一篇：包](#) ->

[English](#) | [日本語](#) | [简体中文](#)

包

[<- 上一篇：高级特性](#) | [下一篇：契约式设计](#) ->

Ry 使用包系统来管理跨文件和目录的代码。详细规格请参阅[包参考手册](#)。

from/import 语法

使用 from 语法导入其他文件的函数。

```
from math import sqrt, PI    # 选择性导入
from math                    # 全部导入（所有定义）
```

这样就可以使用 math.ry 中定义的函数。

子目录（点分隔路径）

使用点分隔路径指定子目录中的包。

```
from utils.calc import add    # 从 utils/calc.ry 导入
```

每个点对应一层目录分隔。

目录包

包可以是单一的 .ry 文件，也可以是包含多个 .ry 文件的目录。当包解析为目录时，其中所有的 .ry 文件会自动加载。

```
mypackage/
  calc.ry      # fn add(), fn sub()
  string.ry    # fn concat()

from mypackage      # 导入 add、sub、concat
from mypackage import add  # 仅导入 add
```

不需要特殊的入口文件（如 __init__.py）。以 _ 开头的文件会被排除。

标准库（std）

std 包会自动导入到所有程序中。不需要编写 from std —— 它始终可用。

```
# 这些函数无需导入即可使用
print("hello")
n = length("world")
xs = range(5)
```

也可以从标准库的包中显式导入特定定义：

```
from str import contains
```

RY_HOME

标准库安装在 \$RY_HOME/lib/std/。RY_HOME 的默认值为 ~/.ry。

```
export RY_HOME="$HOME/.ry"    # 默认
```

搜索路径优先级

包文件按以下顺序搜索：

1. 导入源文件的目录 —— 首先搜索包含导入语句的文件所在的目录。
 2. `$RY_HOME/lib` —— 标准库的位置。
 3. 可执行文件相对的 `lib/` —— 相对于 `ry` 可执行文件的目录。
 4. `RY_PATH` 环境变量 —— 如果未找到，按顺序搜索 `RY_PATH` 中指定的目录。
-

`RY_PATH` 环境变量

可以使用冒号分隔指定多个目录。

```
export RY_PATH=/home/user/ry-libs:/usr/local/ry-libs
```

设置后，可以从任何地方导入指定目录中的包。

限制

- `from` 语句只能写在文件的顶层，不能写在函数或块内部。
- 多次导入同一包时，会自动跳过（不会发生重复导入）。
- 循环导入（A 导入 B，B 导入 A）会产生错误。

```
# 错误示例：a.ry 和 b.ry 互相导入
# a.ry: from b import foo
# b.ry: from a import bar  <- 循环导入错误
```

[<- 上一篇：高级特性](#) | [下一篇：契约式设计](#) ->

[English](#) | [日本語](#) | [简体中文](#)

契约式设计

[<- 上一篇：包](#) | [下一篇：测试](#) ->

Ry 支持 Eiffel 风格的契约式设计，包含前置条件（`require`）、后置条件（`ensure`）以及结构体不变量（`invariant`）。当契约违反时，程序会终止。详细规格请参阅[契约式设计参考手册](#)。

前置条件（`require`）

使用 `require` 指定函数被调用时必须为真的条件。

```
fn deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  new_balance: int = balance + amount
  return new_balance
```

当前置条件不满足时，程序会以下列消息终止：

```
Contract violation: require failed in deposit()
```

后置条件 (ensure)

使用 `ensure` 指定函数返回时必须为真的条件。返回值会绑定到用户选择的变量名。

```
fn abs(x: int) -> int:
  ensure v:
    v >= 0
  if x < 0:
    return -x
  return x
```

由于 `Ry` 的函数参数是不可变的，可以在 `ensure` 块中直接引用参数：

```
fn increment(x: int) -> int:
  ensure v:
    v == x + 1
  return x + 1
```

对于返回元组的函数，可以用逗号分隔指定多个变量名：

```
fn divide(a: int, b: int) -> (int, int):
  ensure q, r:
    q >= 0
    r >= 0
  return (a // b, a % b)
```

同时使用 `require` 和 `ensure`

两者可以同时使用。`require` 必须写在 `ensure` 之前。

```
fn deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  ensure v:
    v >= 0
    v == balance + amount
  new_balance: int = balance + amount
  return new_balance
```

结构体不变量 (invariant)

使用 `invariant` 指定结构体必须始终成立的条件。不变量会在构造后及每次字段赋值后检查。

```
record BankAccount:
  balance: int
  min_balance: int
  invariant:
    balance >= min_balance

a = BankAccount(100, 0)    # OK: 100 >= 0
# a.balance = -1          # Contract violation: invariant failed
```

规则

- `require` 和 `ensure` 块为可选项，写在函数主体之前。
- 同时使用时，`require` 必须写在 `ensure` 之前。
- `ensure` 需要变量绑定来命名返回值（例：`ensure v:`）。
- 对于元组返回值，可指定多个绑定变量（例：`ensure q, r:`）。
- `invariant` 写在 `record` 定义的末尾，在所有字段声明之后。
- 所有契约违反都会以 `exit(1)` 终止程序。

[<- 上一篇：包](#) | [下一篇：测试](#) ->

[English](#) | [日本語](#) | [简体中文](#)

测试

[<- 上一篇：契约式设计](#)

Ry 内置了使用 `describe`、`it`、`expect` 的 RSpec 风格测试语法。详细规格请参阅[测试参考手册](#)。

运行测试

```
ry test                # 自动发现并运行所有 *.test.ry 文件
ry test tests/spec      # 递归运行指定目录下所有 *.test.ry 文件
ry test tests/my_test.test.ry # 运行特定的测试文件
ry test -p              # 并行运行所有测试 (-p 或 --parallel)
```

所有测试通过时，退出码为 0；若有任一测试失败，则为 1。

不带参数运行时，`ry test` 会搜索 `package.toml` 来找到项目根目录，然后递归地发现所有 `*.test.ry` 文件。

编写测试

使用 `describe` 将相关测试分组，使用 `it` 定义各个测试用例。

```
describe("Calculator", fn():
  it("adds integers", fn():
    expect(1 + 2).to_eq(3)
  )
  it("subtracts integers", fn():
    expect(5 - 3).to_eq(2)
  )
  it("checks booleans", fn():
    expect(3 > 1).to_be_true()
  )
)
```

- `describe` 和 `it` 接受描述字符串和 `lambda` 参数 `fn()`：作为第二个参数
- `describe`、`it`、`expect`、`mock` 和 `verify` 仅可在 `ry test` 中使用（普通的 `ry` 执行会产生编译错误）

匹配器

匹配器	说明	支持类型
<code>to_eq(expected)</code>	等值比较	int, float, bool, str
<code>to_not_eq(expected)</code>	不等值断言	int, float, bool, str
<code>to_be_true()</code>	true 断言	bool
<code>to_be_false()</code>	false 断言	bool
<code>to_be_none()</code>	None 断言	Option
<code>to_be_some()</code>	Option 为 Some 的断言	Option
<code>to_contain(val)</code>	容器包含值的断言	List, Set, str

输出格式

```
Calculator
+ adds integers
+ subtracts integers
- checks booleans
  line 10: expected true, got false
```

2 passed, 1 failed

+ 表示通过（绿色），- 表示失败（红色）。

模拟 (Mock)

`mock(fn_name, replacement)`

在当前的 `it` 块中将函数替换为模拟实现。 `it` 块结束时会自动恢复。

```
fn fetch_data() -> str:
  return "real data"

describe("mocking", fn():
  it("replaces function", fn():
    mock(fetch_data, fn() => "fake")
    expect(fetch_data()).to_eq("fake")

  )
  it("auto-restores", fn():
    expect(fetch_data()).to_eq("real data")
  )
)
```

`verify(fn_name)`

返回模拟函数被调用的次数。

```
describe("verify", fn():
  it("counts calls", fn():
    mock(fetch_data, fn() => "fake")
    fetch_data()
    fetch_data()
```

```
        expect(verify(fetch_data)).to_eq(2)
    )
}
```

参数化测试

使用 `@each` 以多组输入运行同一个测试：

```
@each([(1, 2, 3), (0, 0, 0), (-1, 1, 0)])
it("adds {0} + {1} = {2}", fn(a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)
)
```

每个元组成为一个独立的测试用例。描述中的 `{0}`、`{1}` 等会被替换为实际值。

基于属性的测试

使用 `@property` 以随机生成的输入进行测试：

```
@property(count=100)
it("addition is commutative", fn(a: int, b: int):
    expect(a + b).to_eq(b + a)
)
```

测试以随机值运行 `count` 次。失败时会显示反例。

限制

- 不支持 `describe` 的嵌套使用
 - 不支持 `before_each` / `after_each`
 - 重载函数及 `@native` 函数无法模拟
-

[<- 上一篇：契约式设计](#)